THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L6 Model selection

Dr. Zixin Zhong

Data Science and Analytics Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

March 2, 2026

Course feedback

♡ Please scan the QR code and share your feedback.
No worries as it is anonymous. We sincerely appreciate your help.



Syllabus

Lec #	Topic	Lecturer
1	Introduction + course Info	Zixin
2-4	Supervised learning: regression and classification	Zixin
5	Feature engineering + Model evaluation and choice	Zixin
6	**Model evaluation and choice + Feature selection**	Zixin
7	Ensemble learning (supervised learning)	Weikai
	Midterm (28 March, 1:30 - 3:00 PM): mark it on calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	Dimension reduction	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	



Outline

1. Model evaluation and choice
   - Generalization of supervised learning algorithms
   - Error metric, regularizer, no free lunch!
   - Generalization, overfitting
   - Data partition: train, test and validation
2. Feature selection



Recap of Lecture 5

- SVM (partially covered)
  - Linearly separable, γ-linearly separable
  - Margin, minimum margin, geometric margin
  - SVM: maximum margin linear classifier, unique solution
  - SVM: offset, slack, leave-one-out cross-validation error
- Feature engineering
  - Polynomial transformation, nonlinear transformation
  - Boolean, nominal, ordinal, missing values
- Model selection (partially covered)
  - Generalization of supervised learning algorithms
  - Error metric, regularizer, no free lunch!
  - Generalization, overfitting
  - Data partition: train, test and validation



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ means

$$\begin{cases} \boldsymbol{\theta}^\top \mathbf{x} > 0, y = +1 \\ \boldsymbol{\theta}^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow \; y = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}) \; \Leftrightarrow \; y\boldsymbol{\theta}^\top \mathbf{x} > 0.$$

Definition 0.1 (Margin of classifier)

The **margin** of classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\boldsymbol{\theta}^\top \mathbf{x} \text{ or } y\langle \boldsymbol{\theta}, \mathbf{x} \rangle$$

.

- Positive margin: $(\mathbf{x}, y)$ is correctly classified by $\boldsymbol{\theta}$.
- Negative margin: $(\mathbf{x}, y)$ is **not** correctly classified by $\boldsymbol{\theta}$.
- Bigger margin: $(\mathbf{x}, y)$ is **more** correctly classified by $\boldsymbol{\theta}$.



Linearly separable

Definition 0.2 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

$$y_t = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \qquad \forall\, t = 1, 2, \ldots, n. \tag{0.1}$$

There is some hyperplane that (strictly) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min\limits_{1 \le t \le n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$



γ-linearly separable

Definition 0.3 (γ-linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is γ-**linearly separable** for some $\gamma > 0$ if there exists some $\boldsymbol{\theta}$ such that

$$y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle \geq \gamma \qquad \forall\, t = 1, 2, \ldots, n. \tag{0.2}$$

There is some hyperplane that (strictly) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$

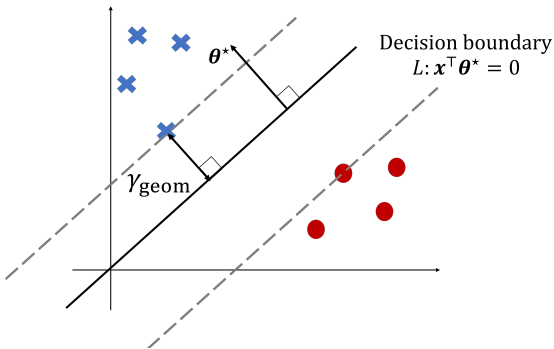


Geometric margin

⊙ **Geometric margin:**

$$\gamma_{\text{geom}} := \frac{\gamma}{\|\boldsymbol{\theta}\|} = \frac{\min\limits_{1\le t\le n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}.$$

- Smallest distance over all samples to the decision boundary $L = \{x \in \mathbb{R}^d : \langle \mathbf{x}, \boldsymbol{\theta} \rangle = 0\}$.
- $\gamma_{\text{geom}}^{-1}$: fair measure of the difficulty of the classification problem





Support Vector Machine (SVM)

- The solution to PRIMAL-SVM is **unique**:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \left\| \boldsymbol{\theta}' \right\|^2 \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$

- SVM algorithm: find the maximum (geometric) margin linear classifier
- Why do we seek the maximum (geometric) margin linear classifier?
- Prove the solution to PRIMAL-SVM is **unique**?



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\boldsymbol{\theta}} = (\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2)/2$
- Check feasibility:

  Since $\forall t = 1, 2, \ldots, n$, $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 \geq 1$ and $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$, by linearity of inner product, we have

$$y_t \mathbf{x}_t^\top \bar{\boldsymbol{\theta}} = \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 + \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$$

  $\Rightarrow \bar{\boldsymbol{\theta}}$ is feasible



Primal-SVM: unique solution

Proof (continued):

$$\left\|\bar{\boldsymbol{\theta}}\right\| = \left\|\frac{1}{2}\left(\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2\right)\right\| \overset{\text{Triangle inequality}}{\leq} \frac{1}{2}\left\|\boldsymbol{\theta}_1\right\| + \frac{1}{2}\left\|\boldsymbol{\theta}_2\right\|$$

$$= \left\|\boldsymbol{\theta}_1\right\| = \left\|\boldsymbol{\theta}_2\right\|$$

Two case:

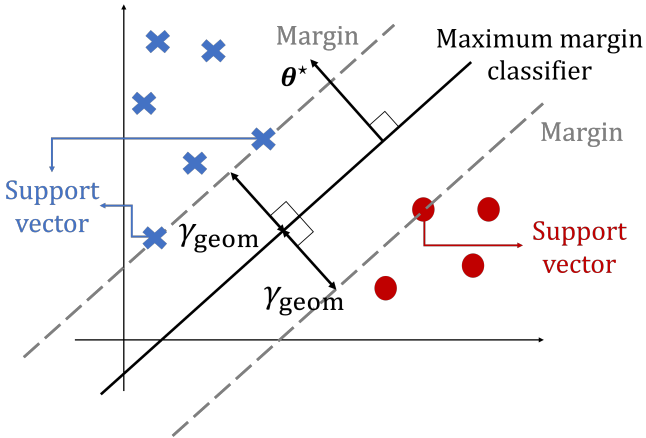
1. $\|\bar{\boldsymbol{\theta}}\| < \|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\| \Rightarrow$ Contradiction!
2. $\|\bar{\boldsymbol{\theta}}\| = \|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\| \Rightarrow$ Triangle inequality holds with equality. This can only happen if $\boldsymbol{\theta}_1$ is in the same direction as $\boldsymbol{\theta}_2$.

In conclusion: there DO NOT exist 2 distinct min length solution vectors to PRIMAL-SVM.



Support Vector + Machine

- Support vector $\Leftarrow$ come from the input data set
  influence the position of the max-margin classifier
- Machine: Input $\Longrightarrow$ Machine $\Longrightarrow$ Output classifier

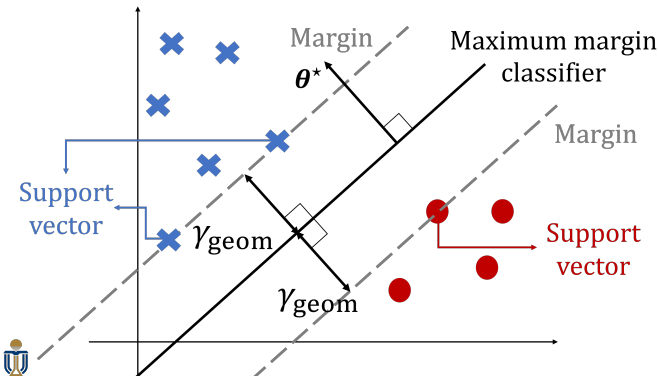


Robustness of max margin classifier

- As we have mentioned: larger margin $\Rightarrow$ more robust
- The (Primal-SVM with Offset) problem:

$$\min_{(\boldsymbol{\theta},\theta)\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{2}\|\boldsymbol{\theta}\|^2 \text{ s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0)\geq 1, \quad \forall t=1,2,\ldots,n$$

- Solution depends on a small number of data samples $(\mathbf{x}_t, y_t)$



Support vectors:

- Samples exactly on the margin.
- Quantify the quality of this binary classifier.

Offset and slack

(Primal-SVM with offset and slack)

$$\min_{\boldsymbol{\theta},\theta_0,\boldsymbol{\xi}\in\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}^n_+} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^n \xi_t$$

$$\text{s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0) \geq 1-\xi_t,\ \xi_t \geq 0 \quad \forall t = 1,2,\ldots,n$$

$$\Longleftrightarrow$$

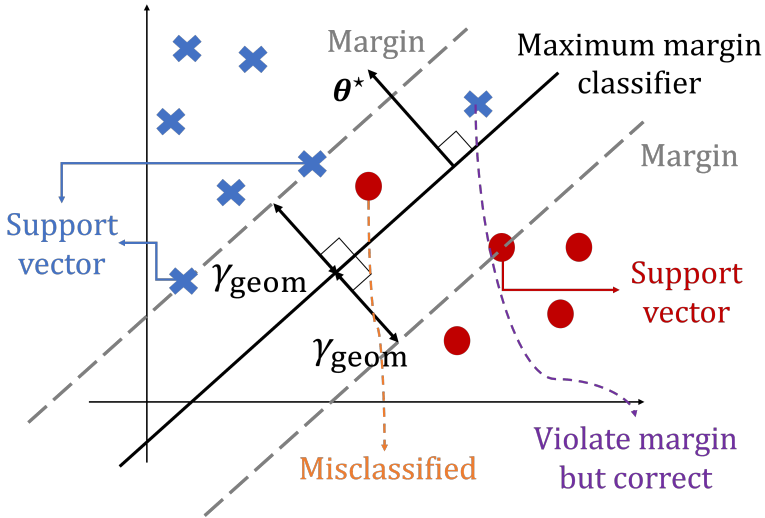
$$\min_{\boldsymbol{\theta},\theta_0\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^n \underbrace{\left[1-y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0)\right]^+}_{:=\xi_t}$$

where $f(z) = (1-z)^+ = \max\{0, 1-z\}$.

- Why offset? Why slack?
- How did we come to this point?



Allowing misclassified example



Leave-one-out cross-validation error (LOOCV)

To assess the robustness of the SVM

$$\mathrm{LOOCV} = \frac{1}{n}\sum_{t=1}^{n} \mathrm{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))),$$

- $(\boldsymbol{\theta}^{-t}, \theta_0^{-t})$ is learnt from the dataset with i^{th} sample left out, i.e., $\mathcal{D}^{-t} = \mathcal{D} \setminus \{\mathbf{x}_t, y_t\}$
- Loss function: $\mathrm{Loss} = \mathbb{1}\{y \neq y'\}$ is zero-one loss

In other words,

$$(\boldsymbol{\theta}^{-t}, \theta_0^{-t}) \in \underset{(\boldsymbol{\theta},\theta)\in\mathbb{R}^d\times\mathbb{R}}{\arg\min} \left\{ \frac{1}{2}\|\boldsymbol{\theta}\|^2 : y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t \in [n] \setminus \{t\} \right\}.$$



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)

Proposition 0.4

Let the number of **support vectors** *be* $N \in \{0, 1, \ldots, n\}$. *Then*

$$\text{LOOCV} \leq \frac{N}{n}.$$

- If the number of support vectors is small, then LOOCV is also small and the SVM works well on new samples



Leave-one-out cross-validation (LOOCV) error

Hint for proof:

- If $\mathbf{x}_t$ is not a support vector, then it cannot influence the solution of the primal SVM.
- Hence,

$$f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, b^{-t})) = y_t, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

and

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) = 0, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

- For any other $\mathbf{x}_t \in$ SV,

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) \leq 1, \quad \forall \mathbf{x}_t \in \text{Support Vectors (SV)}$$



Support Vector Machine (SVM)

- An algorithm that learns the maximum (geometric) margin linear classifier
- The solution is unique
- Incorporating an offset may result in larger geometric margin
- LOOCV to assess the robustness of SVM
- For non-linearly separable data: allow misclassified examples by incorporating a slack

- Solving SVM: Dual form, sequential minimal optimization (SMO)...
- ⊙ Reading: Note of 6-867 (MIT)
  Sec 7.2 and Chapter 12 of book "Mathematics for Machine Learning"



Polynomial tranformation

- Generalized Linear Discriminant Function

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i x_i + \sum_{1 \le i \le j \le d} w_{i,j} x_i x_j + \sum_{1 \le i \le j \le k \le d} w_{i,j,k} x_i x_j x_k$$

- Noting that $x_{l,i}$ is the i-th ($1 \le i \le d$) component of the l-th ($1 \le l \le m$) sample, we can stack this into

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{P}\mathbf{w} = \begin{bmatrix} \mathbf{p}_1^\top \mathbf{w} \\ \vdots \\ \mathbf{p}_m^\top \mathbf{w} \end{bmatrix}$$

and

$$\mathbf{p}_l^\top \mathbf{w} = \begin{bmatrix} 1 & x_{l,1} & \dots & x_{l,d} & \dots & x_{l,i}x_{l,j} & \dots & x_{l,i}x_{l,j}x_{l,k} & \dots \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \\ \vdots \\ w_{i,j} \\ \vdots \\ w_{i,j,k} \\ \vdots \end{bmatrix}$$



Notation: Boolean indicator function*

Define

$$\mathbb{1}(\text{statement}) = \begin{cases} 1 & \text{statement is true} \\ 0 & \text{statement is false} \end{cases}$$

Example:

- $\mathbb{1}(1 > 0) = 1$
- $\mathbb{1}(17 < 15) = 0$



Boolean variables*

- $\mathcal{X} = \{true, false\}$
- Transform to numerical variables:
  let $\phi : \mathcal{X} \to \{0, 1\} : \phi(x) = \mathbb{1}(x)$.

  - Example: bank customer dataset
    $x_1 =$ with checking account?
    $x_1 =$ with saving account?
    $x_1 =$ with credit account?



Boolean expressions*

- $\mathcal{X} = \{\text{true}, \text{false}\}^2$
  $= \{(\text{true}, \text{true}), (\text{true}, \text{false}), (\text{false}, \text{true}), (\text{false}, \text{false})\}.$
- For $x = (x_1, x_2) \in \mathcal{X}$
- Let $\phi(x) = [\mathbb{1}(x_1), \mathbb{1}(x_2), \mathbb{1}(x_1 \text{ and } x_2), \mathbb{1}(x_1 \text{ or } x_2)]$

- Equivalent: polynomials in $[\mathbb{1}(x_1), \mathbb{1}(x_2)]$ span the same space
- Encodes logical expressions!



Nominal values: one-hot encoding*

- Nominal data: e.g., $\mathcal{X} = \{\text{apple}, \text{orange}, \text{banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}),\ \mathbb{1}(x = \text{orange}),\ \mathbb{1}(x = \text{banana})]$$

- One-hot encoding: only one element is non-zero
- Familiar?



Nominal values: one-hot encoding*

- Nominal data: e.g., $\mathcal{X} = \{\text{apple}, \text{orange}, \text{banana}\}$
- Let

$$\phi(x) = [\mathbb{1}(x = \text{apple}),\ \mathbb{1}(x = \text{orange}),\ \mathbb{1}(x = \text{banana})]$$

- One-hot encoding: only one element is non-zero
- Familiar?

  Recall multi-class classification with linear models
- **Extension:** sets,

  e.g. you buy an apple and a banana at the same time



Ordinal values: real encoding/Boolean encoding

- Ordinal data: e.g., $\mathcal{X} = \{\text{Stage I, Stage II, Stage III, Stage IV}\}$
- $\mathcal{Y} = \mathbb{R}$, number of years lived after diagnosis

⊙ Real encoding:

$$\phi(x) = \begin{cases} 1, & x = \text{Stage I} \\ 2, & x = \text{Stage II} \\ 3, & x = \text{Stage III} \\ 4, & x = \text{Stage IV} \end{cases}$$

⊙ Boolean encoding:

$$\phi(x) = [\mathbb{1}(x \geq \text{Stage II}), \mathbb{1}(x \geq \text{Stage III}), \mathbb{1}(x \geq \text{Stage IV})]$$



Missing values

- Remove rows/columns with missing entries
- (For time series) back-fill with most recent observed value
- Impute with mean, median, or mode
- Fancier imputation methods (unfortunately NOT covered in this class): matrix completion, deep learning, ...
- Add new feature: Boolean indicator 1(data is missing)
  - Can detect if missingness is informative
  - Can complement imputation method
  - Can use different indicators for different kinds of missingness (refused, missing, illegible response, ...)



Nonlinear transformations

- Sometimes data is easy to predict with a simple but **nonlinear** relation, e.g.

$$\log(y) = \mathbf{w}^\top \mathbf{x} \text{ equivalent to } y = \exp(\mathbf{w}^\top \mathbf{x})$$

  can transform x or (even more important) y
  - Familiar?

- Nonlinear transforms:

$$\log,\ \exp, \text{quantile}, \text{median}, \ldots$$

- More systematic ways to handle nonlinearities (unfortunately NOT covered here):
  - Deep learning, ...



Location

can be given as

- latitude, longitude
- zip code
- neighborhood, county, state, country

◎ Can be transformed between these!

⊙ Which makes sense for your problem?

- Does nearness matter?
- Are there sharp boundaries? e.g., good district for this school but bad for another one
- Are other properties of the location (e.g., mean house price or crime rate) more important?



Outline

1. Model evaluation and choice
   - Generalization of supervised learning algorithms
   - Error metric, regularizer, no free lunch!
   - Generalization, overfitting
   - Data partition: train, test and validation
2. Feature selection

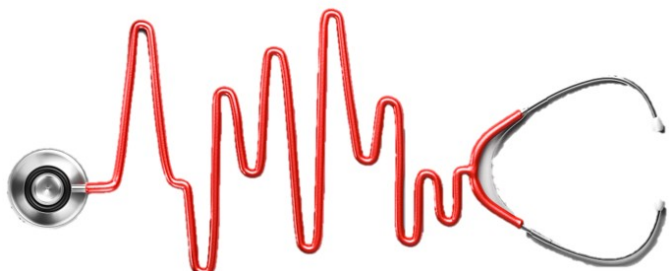


Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.

⊙ What evidence would you ask for to **verify this claim**?



Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.

⊙ What evidence would you ask for to **verify this claim**?

Ideas:

- What data was the system trained on?



Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.

⊙ What evidence would you ask for to **verify this claim**?

Ideas:

- What data was the system trained on?
- How well does it perform on your hospital's data?



Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.

⊙ What evidence would you ask for to **verify this claim**?

Ideas:

- What data was the system trained on?
- How well does it perform on your hospital's data?
- How well does it do on population subgroups? gender, age, race...



Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.
⊙ What evidence would you ask for to **verify this claim**?

Ideas:

- What data was the system trained on?
- How well does it perform on your hospital's data?
- How well does it do on population subgroups? gender, age, race...
- How many false positives / false negatives?



Motivation

You run a hospital. A vendor wants to sell you a new machine learning system for diagnosing colon cancer. The vendor tells you the system works with 99% accuracy.

You stand to save millions of dollars and hundreds of lives by catching the disease early.
⊙ What evidence would you ask for to **verify this claim**?

Ideas:

- What data was the system trained on?
- How well does it perform on your hospital's data?
- How well does it do on population subgroups? gender, age, race...
- How many false positives / false negatives?
- What is the accuracy on true positives/negatives?



Generalization of supervised learning algorithms

- We need the new ML system generalizes well on unseen data, e.g., data from my hospital or other hospitals



Generalization of supervised learning algorithms

- We need the new ML system generalizes well on unseen data, e.g., data from my hospital or other hospitals

- Elements in supervised learning:
  - Unknown target function $f : \mathcal{X} \to \mathcal{Y}$
  - Training examples $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$



Generalization of supervised learning algorithms

- We need the new ML system generalizes well on unseen data, e.g., data from my hospital or other hospitals

- Elements in supervised learning:
  - Unknown target function $f : \mathcal{X} \to \mathcal{Y}$
  - Training examples $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$
  - Hypothesis set $\mathcal{H}$ (set of functions)
  - Learning algorithm $\mathcal{A}$
  - Final hypothesis $g : \mathcal{X} \to \mathcal{Y}$



Generalization of supervised learning algorithms

- We need the new ML system generalizes well on unseen data, e.g., data from my hospital or other hospitals

- Elements in supervised learning:
  - Unknown target function $f : \mathcal{X} \to \mathcal{Y}$
  - Training examples $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$
  - Hypothesis set $\mathcal{H}$ (set of functions)
  - Learning algorithm $\mathcal{A}$
  - Final hypothesis $g : \mathcal{X} \to \mathcal{Y}$

- How well will our classifier/regressor do on new data?
  $\Leftarrow$ how well can g approximate f?



A cautionary tale: predict who will be approved for credit card

- Dataset $\mathcal{D}$ = all previous $\underbrace{\text{applicants}}_{x_i}$ + $\underbrace{\text{decisions}}_{y_i}$
- Hypothesis set $\mathcal{H}$ = rules of the form:

  pick a subset of names from $\mathcal{D}$, and predicts 1 if applicant is on the subset, i.e.,

$$h(\text{applicant}) = \begin{cases} 1 & \text{applicant name} = \text{Alpha Adams} \\ 1 & \text{applicant name} = \text{Beta Betty} \\ 1 & \dots \\ -1 & \text{otherwise} \end{cases}$$

- Learning algorithm $\mathcal{A}$: pick h that performs best on $\mathcal{D}$



A cautionary tale: predict who will be approved for credit card

- Dataset $\mathcal{D}$ = all previous $\underbrace{\text{applicants}}_{x_i}$ + $\underbrace{\text{decisions}}_{y_i}$
- Hypothesis set $\mathcal{H}$ = rules of the form:

  pick a subset of names from $\mathcal{D}$, and predicts 1 if applicant is on the subset, i.e.,

$$h(\text{applicant}) = \begin{cases} 1 & \text{applicant name} = \text{Alpha Adams} \\ 1 & \text{applicant name} = \text{Beta Betty} \\ 1 & \dots \\ -1 & \text{otherwise} \end{cases}$$

- Learning algorithm $\mathcal{A}$: pick h that performs best on $\mathcal{D}$

Q: Final hypothesis h classifies $X\%$ of the dataset correctly. Guess the value of X?

A. 0 B. 50 C. 100



A cautionary tale: predict who will be approved for credit card

- Dataset $\mathcal{D}$ = all previous $\underbrace{\text{applicants}}_{x_i}$ + $\underbrace{\text{decisions}}_{y_i}$
- Hypothesis set $\mathcal{H}$ = rules of the form:

  pick a subset of names from $\mathcal{D}$, and predicts 1 if applicant is on the subset, i.e.,

$$h(\text{applicant}) = \begin{cases} 1 & \text{applicant name} = \text{Alpha Adams} \\ 1 & \text{applicant name} = \text{Beta Betty} \\ 1 & \dots \\ -1 & \text{otherwise} \end{cases}$$

- Learning algorithm $\mathcal{A}$: pick h that performs best on $\mathcal{D}$
- How? $\Rightarrow$ final hypothesis h memorizes the data

$$h(\text{applicant}) = \begin{cases} 1 & \text{applicant} = x_i \text{ and } y_i = 1 \text{ for some } i \\ -1 & \text{otherwise} \end{cases}$$



Generalization of supervised learning algorithm: key elements

- Unknown target function $f : \mathcal{X} \to \mathcal{Y}$
- Training examples $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$
- Hypothesis set $\mathcal{H}$ (set of functions)
- Learning algorithm $\mathcal{A}$
- Final hypothesis $g : \mathcal{X} \to \mathcal{Y}$



Linear regression: key elements

- **Learning/Training**: Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$.
  - Design an affine model such that $f_{\overline{\mathbf{w}}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = y$ where $\overline{\mathbf{w}} = [\mathbf{w}^\top \ b]^\top$.
  - Find the minimizer of loss function

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} (f_{\overline{\mathbf{w}}}(\mathbf{x}_i) - y_i)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}).$$

  - Least squares solution (with offset)

$$\overline{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}.$$

- **Prediction/Testing**: Given a new feature vector (sample, example) $\mathbf{x}_{\text{new}}$, the prediction based on the least squares solution is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^* = b^* + \mathbf{x}_{\text{new}}^\top \mathbf{w}^*.$$



Linear regression: key elements

- **Learning/Training**: Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$.
  - (Hypothesis class) Design an affine model such that $f_{\overline{\mathbf{w}}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = y$ where $\overline{\mathbf{w}} = [\mathbf{w}^\top \ b]^\top$.
  - (Learning algorithm) Find the minimizer of loss function

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} (f_{\overline{\mathbf{w}}}(\mathbf{x}_i) - y_i)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}).$$

  - (Final hypothesis) Least squares solution (with offset)

$$\overline{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}.$$

- **Prediction/Testing**: Given a new feature vector (sample, example) $\mathbf{x}_{\text{new}}$, the prediction based on the least squares solution is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^* = b^* + \mathbf{x}_{\text{new}}^\top \mathbf{w}^*.$$



Performance estimation

- **Q:** How well will our model (classifier) do on new data?
  - A. Terribly
  - B. Okay
  - C. Extremely well
  - D. Cannot tell



Performance estimation

- **Q:** How well will our model (classifier) do on new data?

A. Terribly

B. Okay

C. Extremely well

D. Cannot tell

- **Q:** How to predict performance of the model on new data?
  - (Given infinite data) evaluate model on new data
  - (Given finite data) split data; fit on one part, evaluate on another part



Performance estimation

- **Q:** How well will our model (classifier) do on new data?
  A. Terribly
  B. Okay
  C. Extremely well
  D. Cannot tell

- **Q:** How to predict performance of the model on new data?
  - (Given infinite data) evaluate model on new data
  - (Given finite data) split data; fit on one part, evaluate on another part



Data partition: train, test and validation



- Fit model to the **training** dataset
- Fit hyperparameters to the **validation** dataset
- Test model performance on the **test** dataset



Error metric

Q: How to measure how well model fits?

A: Define an **error metric**: **error:** $\mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$

$$\textbf{error}(y, y') = \text{penalty for predicting } y' \text{ when true label is } y$$

Choose an error metric that **makes sense** for your application!



Error metric: some candidates

- **Regression**
  - Square error: $\textbf{error}_{\mathrm{sq}}(y, y') = (y - y')^2$
  - Absolute error: $\textbf{error}\mathrm{abs}(y, y') = |y - y'|$



Error metric: some candidates

Regression

- Square error: $\textbf{error}_{\text{sq}}(y, y') = (y - y')^2$
- Absolute error: $\textbf{error}\text{abs}(y, y') = |y - y'|$

Classification

- Misclassification error: $\textbf{error}_{\text{mis}}(y, y') = \mathbb{1}\{y \neq y'\}$
- Weighted misclassification error: If false positives are β times worse than false negatives

$$\textbf{error}_{\text{beta}}(y, y') = \beta \mathbb{1}\{y' = 1, y = -1\} + (1 - \beta)\mathbb{1}\{y' = -1, y = 1\}$$

- Balanced error rate: If dataset contains n_+ examples with positive labels and n_- negative

$$\textbf{error}_{\text{bal}}(y, y') = \frac{\mathbb{1}\{y \neq y'\}}{n_+ \mathbb{1}\{y\} + n_- \mathbb{1}\{1 - \mathbb{1}\{y\}\}}$$



Confusion matrix

		Predicted condition	
	Total population = P+N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)



Confusion matrix

		Predicted condition	
	Total population = P+N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)
		Precision: $\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$	Accuracy: $\frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{TN}}$



Some ways to further evaluate classification model

- Precision = # true pos / (# true pos + # false pos)
  - Measure the accuracy of positive predictions



Some ways to further evaluate classification model

- Precision = # true pos / (# true pos + # false pos)
  - Measure the accuracy of positive predictions

- Recall (sensitivity) = # true pos / (# true pos + # false neg)
  - Measures the model's ability to find all the positive instances



Some ways to further evaluate classification model

- Precision = # true pos / (# true pos + # false pos)
  - Measure the accuracy of positive predictions

- Recall (sensitivity) = # true pos / (# true pos + # false neg)
  - Measures the model's ability to find all the positive instances

- F1 score = $2/(\text{Recall}^{-1}+\text{Precision}^{-1})$
  - Harmonic mean of precision and recall: balance the two metrics into a single number, making it especially useful when precision and recall are in trade-off



Exercise

Given the following classification result:
Ground truth label: `A,A,A,A,A,B,B,B,B,B`
Predictions: `A,A,A,A,B,B,B,A,A,A`

Calculate the precision and recall

A 4/7, 4/5

B 4/5, 4/7

C 4/6, 6/10

D 6/10, 4/7

E None of the above



Some ways to further evaluate classification model

- True Positive Rate = # true pos / (# true pos + # false neg)
  - Measure the accuracy of prediction for positive samples



Some ways to further evaluate classification model

- True Positive Rate = # true pos / (# true pos + # false neg)
  - Measure the accuracy of prediction for positive samples

- False Positive Rate = # false pos / (# false pos + # true neg)
  - Measure the accuracy of prediction for negative samples



Some ways to further evaluate classification model

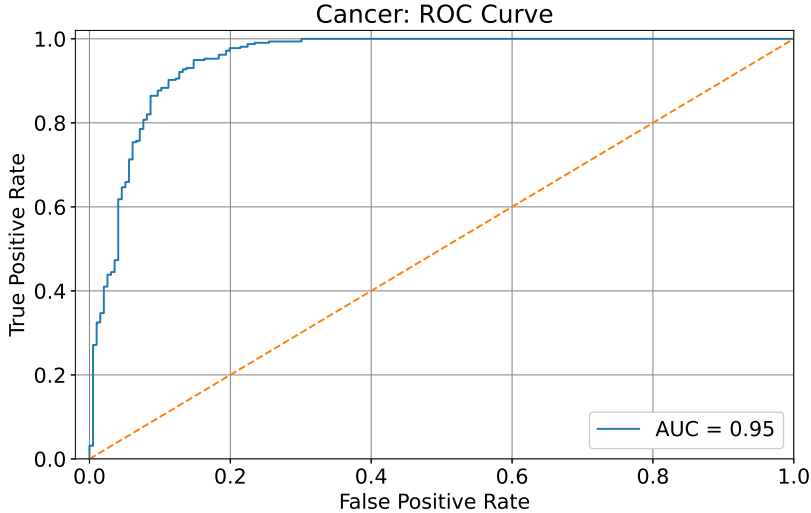
- True Positive Rate = # true pos / (# true pos + # false neg)
  - Measure the accuracy of prediction for positive samples

- False Positive Rate = # false pos / (# false pos + # true neg)
  - Measure the accuracy of prediction for negative samples

- ROC, AUC (AUROC)
  - ROC: Receiver Operating Characteristic
    AUC (AUROC): Area Under the Receiver Operating Characteristic
  - Remove dependence on thresholds

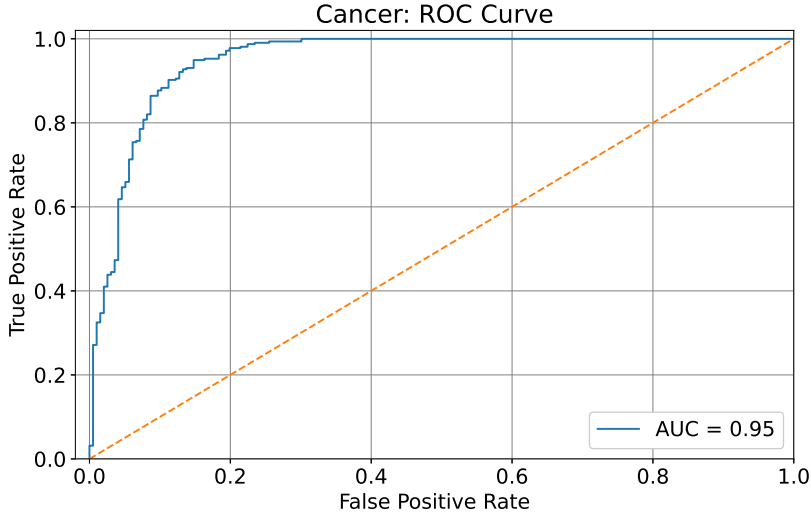


Demo: ROC, AUC


Cancer: ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
AUC = 0.95



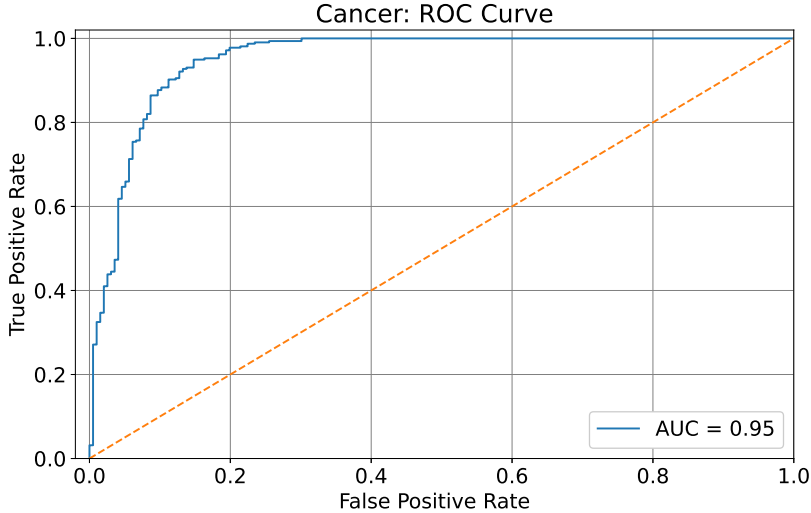
Demo: ROC, AUC



Q: Given a set of models with different AUC: $0.24, 0.52, 0.9, 1$, which is the best model?



Demo: ROC, AUC



⊙ Canvas: `DSAA2011-L06_model_sel_demo_roc.pdf`



Some ways to further evaluate classification model

- Multiclass?
  - Calculate precision, recall within each class
  - Average them across the classes
  - Consider balanced average?

- Python:
  - `sklearn.metrics.average_precision_score`
  - `sklearn.metrics.recall_score`
  - `sklearn.metrics.precision_score`
  - `sklearn.metrics.f1_score`
  - `sklearn.metrics.roc_curve`
  - `sklearn.metrics.auc`



Train and test error: evaluate the selected error metric on dataset

- Partition data $\mathcal{D}$ into training set $\mathcal{D}_{\text{train}}$ and test set $\mathcal{D}_{\text{test}}$

  $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$, $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}$
- Algorithm $\mathcal{A}$ is only allowed to see the training set $\mathcal{D}_{\text{train}}$



Train and test error: evaluate the selected error metric on dataset

- Partition data $\mathcal{D}$ into training set $\mathcal{D}_{\text{train}}$ and test set $\mathcal{D}_{\text{test}}$

  $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$, $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}$
- Algorithm $\mathcal{A}$ is only allowed to see the training set $\mathcal{D}_{\text{train}}$
- **Training error**

$$E_{\text{train}}(g) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \text{error}(y_i, g(x_i))$$

  Test error

$$E_{\text{test}}(g) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} \text{error}(y_i, g(x_i))$$

  - Mean Error: "Mean Square Error" (MSE), "Mean Absolute Error" (MAE), etc.



Q: How to measure how well model fits?

A: Define an **error metric** . . . . . .



Q: How to measure how well model fits?

A: Define an **error metric** . . . . . .

Q: If the error is quite small, can we say the model/hypothesis is good already?



Recap: ridge regression

- **Ridge regression:** For a fixed $\lambda \geq 0$, consider

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i\right)^2 + \lambda \sum_{i=0}^{d} w_j^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}$$

- The term $\lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}$ encourages the weight vector to have small components (also known as shrinkage.
- When $\lambda = 0$, we recover usual linear regression.



Recap: logistic regression

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

  - Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - Regularization: e.g., ℓ_2-norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^{n} \log \left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{1.1}$$



Insight

$$\text{Objective function} = \text{Error} + \text{Regularizer}$$



Regularization

◎ **Why regularize?**

- Reduce variance of the model (more stable)
- Prevent overfitting
- Impose prior structural knowledge
- Improve interpretability: simpler model, automatic feature selection



Regularization

- **Why regularize?**
  - Reduce variance of the model (more stable)
  - Prevent overfitting
  - Impose prior structural knowledge
  - Improve interpretability: simpler model, automatic feature selection
- **Why NOT regularize?**
  - Gauss-Markov theorem:
    - Least squares is the best linear unbiased estimator
  - Regularization increases bias



Regularization

◎ **Why regularize?**

- Reduce variance of the model (more stable)
- Prevent overfitting
- Impose prior structural knowledge
- Improve interpretability: simpler model, automatic feature selection

◎ **Why NOT regularize?**

- Gauss-Markov theorem:
  - Least squares is the best linear unbiased estimator
- Regularization increases bias

Any question here?



Regularization

◎ **Why regularize?**

- Reduce variance of the model (more stable)
- Prevent overfitting
- Impose prior structural knowledge
- Improve interpretability: simpler model, automatic feature selection

◎ **Why NOT regularize?**

- Gauss-Markov theorem:
  - Least squares is the best linear unbiased estimator
- Regularization increases bias

Q: What is variance? What is bias?



No free lunch!

Decompose the expected error:

$$\text{expected prediction error} = \text{bias}^2 + \text{variance} + \text{noise}$$

- bias^2: average prediction error over all data sets (squared)
- variance: how much solutions for different data sets vary (stability)
- noise: deviation of measurements from the true value (unavoidable error)



No free lunch!

Decompose the expected error:

$$\text{expected prediction error} = \text{bias}^2 + \text{variance} + \text{noise}$$

- bias^2: average prediction error over all data sets (squared)
- variance: how much solutions for different data sets vary (stability)
- noise: deviation of measurements from the true value (unavoidable error)

Theorem 1.1 (No free lunch!)

Problem: *Low bias comes with high variance and low variances comes with high bias!*

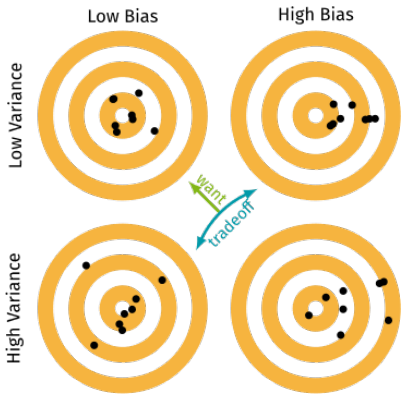
- *Too high bias: we cannot fit the data well, because our solutions are too restricted.*
- *Too low bias: we have too high variance, sensitive to data changes (overfitting).*

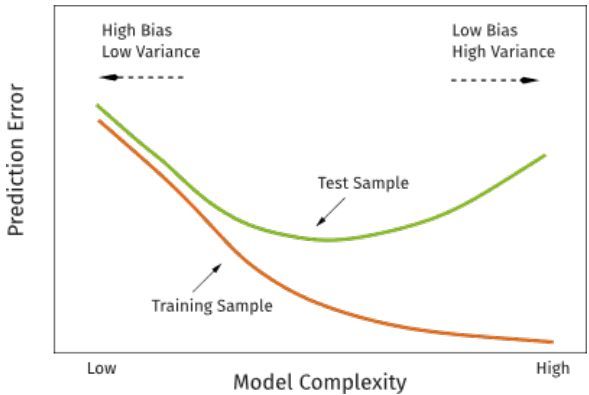
$\rightarrow$ *We need to find the sweet spot inbetween!*



No free lunch!

⊙ Bias-variance trade-off





No free lunch!

References:

- Section 8 in `https://cs229.stanford.edu/lectures-spring2022/main_notes.pdf`
- Webpage: `https://dm.cs.tu-dortmund.de/mlbits/foundations-bias-variance/#ref-DBLP:books/lib/Bishop07`
- Book: Bishop, C.M. 2007. Pattern recognition and machine learning, 5th edition. Springer.



Regularization

- **Why regularize?**
  - Reduce variance of the model (more stable)
  - Prevent overfitting
  - Impose prior structural knowledge
  - Improve interpretability: simpler model, automatic feature selection
- **Why NOT regularize?**
  - Gauss-Markov theorem:
    - Least squares is the best linear unbiased estimator
  - Regularization increases bias



Regularizer

We might choose regularizer so models will be

- small
- sparse
- nonnegative
- smooth
- ...



ℓ_p-norm of vector

Definition 1.2 (ℓ_p-norm of vector)

Let $p \in (0, \infty)$, ℓ_p-norm of $\mathbf{w} \in \mathbb{R}^d$ is defined as $||\mathbf{w}||_p = \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p}$.



ℓ_p-norm of vector

Definition 1.2 (ℓ_p-norm of vector)

Let $p \in (0, \infty)$, ℓ_p-norm of $\mathbf{w} \in \mathbb{R}^d$ is defined as $||\mathbf{w}||_p = \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p}$.

- $0 < \ell < \infty$:
  - ℓ_1-norm is $||\mathbf{w}||_1 = \sum_{i=1}^{d} |w_i|$. ℓ_2-norm is $||\mathbf{w}||_2 = \sqrt{\sum_{i=1}^{d} |w_i|^2}$.



ℓ_p-norm of vector

Definition 1.2 (ℓ_p-norm of vector)

Let $p \in (0, \infty)$, ℓ_p-norm of $\mathbf{w} \in \mathbb{R}^d$ is defined as $||\mathbf{w}||_p = \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p}$.

- $0 < \ell < \infty$:
  - ℓ_1-norm is $||\mathbf{w}||_1 = \sum_{i=1}^{d} |w_i|$. ℓ_2-norm is $||\mathbf{w}||_2 = \sqrt{\sum_{i=1}^{d} |w_i|^2}$.
- $p = 0$ or ∞, ℓ_p-norm is defined by taking limit:
  - ℓ_∞-norm is $||\mathbf{w}||_\infty = \lim_{p \to \infty} \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p} = \max_i |w_i|$.
  - ℓ_0-norm is $||\mathbf{w}||_0 = \lim_{p \to 0} \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p} = \sum_{i=1}^{d} \mathbb{1}(w_i \neq 0)$.



ℓ_p-norm of vector

Definition 1.2 (ℓ_p-norm of vector)

Let $p \in (0, \infty)$, ℓ_p-norm of $\mathbf{w} \in \mathbb{R}^d$ is defined as $||\mathbf{w}||_p = \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p}$.

- $0 < \ell < \infty$:
  - ℓ_1-norm is $||\mathbf{w}||_1 = \sum_{i=1}^{d} |w_i|$. ℓ_2-norm is $||\mathbf{w}||_2 = \sqrt{\sum_{i=1}^{d} |w_i|^2}$.
- $p = 0$ or ∞, ℓ_p-norm is defined by taking limit:
  - ℓ_∞-norm is $||\mathbf{w}||_\infty = \lim_{p \to \infty} \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p} = \max_i |w_i|$.
  - ℓ_0-norm is $||\mathbf{w}||_0 = \lim_{p \to 0} \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p} = \sum_{i=1}^{d} \mathbb{1}(w_i \neq 0)$.
- Technical note: ℓ_0 is not actually a norm



(not absolutely homogeneous since $||\alpha \mathbf{w}||_0 = ||\mathbf{w}||_0$ for $\alpha \neq 0$)

ℓ_1 regularization

ℓ_1 regularizer ($\lambda \geq 0$: regularization parameter)

$$r(\mathbf{w}) = \lambda \sum_{i=1}^{d} |w_i| \tag{1.2}$$

Lasso problem: linear least squares + ℓ_1 regularizer

$$\min_{\overline{\mathbf{w}}} \sum_{i=1}^{m} (y_i - \overline{\mathbf{w}}^\top x_i)^2 + \lambda \sum_{i=1}^{d} |\bar{w}_i|$$

with variable $\overline{\mathbf{w}} \in \mathbb{R}^{d+1}$.



ℓ_1 regularization

ℓ_1 regularizer ($\lambda \geq 0$: regularization parameter)

$$r(\mathbf{w}) = \lambda \sum_{i=1}^{d} |w_i| \tag{1.2}$$

Lasso problem: linear least squares + ℓ_1 regularizer

$$\min_{\overline{\mathbf{w}}} \sum_{i=1}^{m} (y_i - \overline{\mathbf{w}}^\top x_i)^2 + \lambda \sum_{i=1}^{d} |\bar{w}_i|$$

with variable $\overline{\mathbf{w}} \in \mathbb{R}^{d+1}$.

- Penalize large $\overline{\mathbf{w}}$ less than quadratic (ℓ_2) regularization: $r(\mathbf{w}) = \lambda ||w||_2^2 = \lambda \sum_{i=1}^{d} w_i^2$
- No closed form solution



ℓ_1 regularization

Why use ℓ_1-norm?

- Tends to produce sparse solution (some coordinates=0)

  e.g. Automatic feature selection, reduces complexity of the model



Generalization and overfitting

- Goal of model is **not** to predict only well on $\mathcal{D}$
- Goal of model is to predict well on **all data (old, new and unseen)**

◦ If the model has ?? training set error and ?? test set error, we say the model:

	Low test set error	**High** test set error
Low training set error	Generalize	Overfit
High training set error	?!?!	Underfit



Poll

⊙ How to fix underfitting?

A. Use more complex model
B. Use less complex model
C. Add new features
D. Remove features
E. Find more data



Poll

⊙ How to fix overfitting?

A. Use more complex model
B. Use less complex model
C. Add new features
D. Remove features
E. Find more data



Poll

⊙ Is it possible to overfit and underfit at the same time?

A. Yes

B. No



Poll

⊙ Is it possible to overfit and underfit at the same time?

A. Yes

B. No

e.g. Training error can be high but still much lower than test error



How to choose train and test tests

- At random (e.g., toss random coin with probability p)
- More data in the training set improves the model fit
- More data in test set helps determine how well the model will work out of sample
- Rule of thumb: put about 20% of data into the test set
- ...

  (`https://scikit-learn.org/stable/api/sklearn.model_selection.html`)



Validation

Training set error improves with model complexity.



Validation

Training set error improves with model complexity.

How to decide which model to use? How to tune hyperparameters?



Validation

Training set error improves with model complexity.

How to decide which model to use? How to tune hyperparameters?

$\Rightarrow$ hold out a validation set



Validation

- A simple and effective validation procedure:
  1. Split data into training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{valid}}$
  2. Pick m different interesting model classes

     e.g., different model class ϕ: $\phi_1, \phi_2, \ldots, \phi_m$
  3. Fit ("train") models on training set $\mathcal{D}_{\text{train}}$

     Get one hypothesis $h : \mathcal{X} \to \mathcal{Y}$ for each ϕs, and set

     $$\mathcal{H} = \{h_1, h_2, \ldots, h_m\}$$
  4. Compute error of each hypothesis on validation set $\mathcal{D}_{\text{valid}}$ and choose the lowest:

     $$h^* = \arg\min_{h \in \mathcal{H}} E_{\mathcal{D}_{\text{valid}}}(h)$$

  - OR: select the model class according to h^* and retrain h^* on the entire training set.



Exercise: Polynomial regression (validation)

Recall the polynomial regression model

How can we choose the best order of polynomial transformation?



Cross-validation

1. Pick a bunch of interesting model classes (e.g., different ϕs: $\phi_1, \phi_2, \ldots, \phi_m$)
2. For each possible split of data into training set $\mathcal{D}$ and validation set $\mathcal{D}'$:

$$(\mathcal{D}_1, \mathcal{D}'_1), (\mathcal{D}_2, \mathcal{D}'_2), \ldots, (\mathcal{D}_K, \mathcal{D}'_K)$$

- Fit ('train') each model (class) ϕ on training set $\mathcal{D}$ and obtain hypothesis $h_{\phi,\mathcal{D}}$
- Compute error of each hypothesis on validation set $\mathcal{D}'$: $E_{\mathcal{D}'}(h_{\phi,\mathcal{D}})$
- Now, the validation set $\mathcal{D}'$ is not a single set
  - $\Rightarrow$ multiple validation sets $\mathcal{D}'$s

3. Estimate error of model as average of error on each $\mathcal{D}'$

$$\phi^* = \arg\min_{\phi_i} \frac{1}{K} \sum_{j=1}^{K} E_{\mathcal{D}'_j}(h_{\phi_i,\mathcal{D}_j})$$

4. Fit the model ϕ^* using the entire dataset.



How to pick splits

- **Leave-one-out cross-validation**
  - $\mathcal{D}'$ has one example, $\mathcal{D}$ has all the rest
    - Advantage: accurate
    - Disadvantage: slow
    - Recall: measure the robustness of SVM

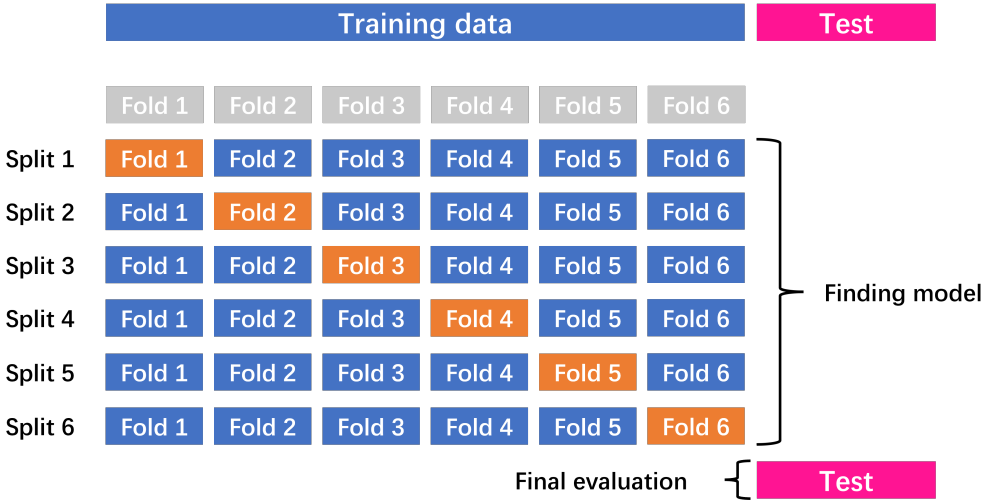


How to pick splits

- **Leave-one-out cross-validation**
  - $\mathcal{D}'$ has one example, $\mathcal{D}$ has all the rest
    - Advantage: accurate
    - Disadvantage: slow
    - Recall: measure the robustness of SVM
- **n-fold cross-validation**
  - $\mathcal{D}'$ has $1/n$ of the entire training dataset, $\mathcal{D}$ has all the rest (usually, $n = 5$ or 10)
    - Decently accurate, not too slow



n-fold cross-validation: example with $n = 6$



Summary: train, test, validation

- We care how well model performs on both old and new data
- "To simulate new data", split $\mathcal{D}$ into train, validation and test sets
- Evaluate error on each validation set via error metric
- If training set error is high, we say model fits poorly
  - Solution: use more complex model, or add new features, or add data
- If test set error is much higher than training set error, we say model overfits
  - Solution: make less complex model, or remove features, or add data

- Reading material: Section 9 in `https://cs229.stanford.edu/lectures-spring2022/main_notes.pdf`



Outline

1. Model evaluation and choice
   - Generalization of supervised learning algorithms
   - Error metric, regularizer, no free lunch!
   - Generalization, overfitting
   - Data partition: train, test and validation

2. Feature selection



What is feature selection?

- Choosing a subset of relevant features from your dataset.



What is feature selection?

- Choosing a subset of relevant features from your dataset.

- **Why?**
- Improve accuracy, reduce overfitting, speed up training.



What is feature selection?

- Choosing a subset of relevant features from your dataset.

- **Why?**
- Improve accuracy, reduce overfitting, speed up training.

- Very important in machine learning
  - e.g., ensemble methods like bagging and boosting!



Why feature selection matters

- **Too many features**: Noise, redundancy, slow models.
- **Too few**: Miss important patterns.



Why feature selection matters

- **Too many features**: Noise, redundancy, slow models.
- **Too few**: Miss important patterns.

- **Goal:** Find the "sweet spot" for better performance.



Why feature selection matters

- **Too many features**: Noise, redundancy, slow models.
- **Too few**: Miss important patterns.

- **Goal:** Find the "sweet spot" for better performance.

- e.g., Ensembles handle many features, but selection makes them shine.



Types of feature selection

- **Filter:** Rank features (e.g., correlation) and abandon a subset of features before modeling.
- **Wrapper:** Test feature subsets with a model (e.g., sequentially add/drop features).
- **Embedded:** Selection during training (e.g., feature importance).
  - Bagging and boosting often use embedded methods!



Filter methods

- Rank features and abandon a subset of features before modeling using statistical methods

- **How?**
  - Variation of feature: variance
  - Relevance between feature and target: Person correlation (numeric), Chi-squared (categorical), mutual information (numeric/categorical)
  - Or using some prior knowledge

- **Pros:** Fast, simple, scalable.
- **Cons:** Miss feature interactions.



Filter methods: variance

- Dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^N$, each sample $\mathbf{x}_t = (x_{t,1}, \ldots, x_{t,d})$
  - Variance of numerical feature:

$$\overline{x_{(i)}} = \frac{1}{N}\sum_{t=1}^{N} x_{t,i}, \ \mathrm{Var}^{(i)} = \frac{1}{N}\sum_{t=1}^{N} \left(x_{t,i} - \overline{x_{(i)}}\right)^2$$

  - Keep feature $\{x_{t,i}\}_{t=1}^N$ with high variance



Filter methods: relevance between feature and target

- Dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^N$, each sample $\mathbf{x}_t = (x_{t,1}, \ldots, x_{t,d})$
  - Pearson Correlation between continuous variables

$$r^{(i)} = \frac{\sum_{i=t}^{n} \left(x_{t,i} - \overline{x_{(i)}}\right)\left(y_t - \bar{y}\right)}{\sqrt{\sum_{i=t}^{n} \left(x_{t,i} - \overline{x_{(i)}}\right)^2} \sqrt{\sum_{t=1}^{n} \left(y_t - \bar{y}\right)^2}}$$

  - Keep feature $\{x_{t,i}\}_{t=1}^N$ with high correlation



Filter methods: relevance between feature and target

- Dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^N$, each sample $\mathbf{x}_t = (x_{t,1}, \ldots, x_{t,d})$
- Chi-squared between categorical variables:
  - $\{x_{t,i}\}_{t=1}^N$ take values in $\{a_1, a_2, \ldots\}$
    $\{y_t\}_{t=1}^N$ take values in $\{b_1, b_2, \ldots\}$
  - $P_j = |\{t : x_{t,i} = a_j\}|$, $Q_k = |\{t : y_t = b_k\}|$
    $O_{jk} = |\{t : x_{t,i} = a_j, y_t = b_k\}|$

$$\chi^2_{(i)} = \sum_{j,k} \frac{(O_{jk} - E_{jk})^2}{E_{jk}}, \quad E_{jk} = \frac{P_j \cdot Q_k}{n}.$$

  - Higher $\chi^2_{(i)} \Rightarrow$ Smaller p-value (< 0.05?) $\Rightarrow$ Correlated!
  - Keep feature $\{x_{t,i}\}_{t=1}^N$ with small p-value



Filter methods: relevance between feature and target

- Dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^N$, each sample $\mathbf{x}_t = (x_{t,1}, \ldots, x_{t,d})$
- mutual information between variables:
  - $\{x_{t,i}\}_{t=1}^N$ take values in the set X
    $\{y_t\}_{t=1}^N$ take values in the set Y

$$\text{Discrete: } I(X;Y) = \sum_{y\in Y}\sum_{x\in X} p(x,y)\log\left(\frac{p(x,y)}{p(x)p(y)}\right)$$

$$\text{Continuous: } I(X;Y) = \int_Y\int_X p(x,y)\log\left(\frac{p(x,y)}{p(x)p(y)}\right)\mathrm{d}x\mathrm{d}y$$

  - Higher $I(X;Y) \Rightarrow$ Correlated!
  - Keep feature $\{x_{t,i}\}_{t=1}^N$ with large $I(X;Y)$



Wrapper methods

- Test feature subsets by training a model.

- **How?**
  - Sequential Forward Selection (SFS): add features
  - Recursive Feature Elimination (RFE): remove features



Wrapper methods

- Test feature subsets by training a model.

- **How?**
  - Sequential Forward Selection (SFS): add features
  - Recursive Feature Elimination (RFE): remove features
- **Example** of SFS: Try "size", then "size" + "location" $\Rightarrow$ pick best accuracy
- **Example** of RFE: Try "temperature" + "location" then "temperature" $\Rightarrow$ pick best accuracy



Wrapper methods

- Test feature subsets by training a model.

- **How?**
  - Sequential Forward Selection (SFS): add features
  - Recursive Feature Elimination (RFE): remove features
- **Example** of SFS: Try "size", then "size" + "location" $\Rightarrow$ pick best accuracy
- **Example** of RFE: Try "temperature" + "location" then "temperature" $\Rightarrow$ pick best accuracy

- **Pros:** Model-specific, considers interactions
- **Cons:** Slow, can overfit to training data



Embedded methods

- Selection during model training.



Embedded methods

- Selection during model training.
- **How:**
  - Use regularizer to penalize unimportant features, e.g., Lasso, Ridge.
  - Decision trees naturally perform feature selection during training.



Embedded methods

- Selection during model training.
- **How:**
  - Use regularizer to penalize unimportant features, e.g., Lasso, Ridge.
  - Decision trees naturally perform feature selection during training.
- **Pros:** Efficient, tailored to model.
- **Cons:** Depends on model choice.



Embedded methods: feature importance in tree models

- Common ideas:
  - *Impurity/Gain*: How much a feature improves splits or reduces error.
  - *Usage*: How often a feature is picked.
  - *Coverage*: How many samples are affected by the split



Thanks for listening

1. Tell us your question/feedback via the QR code.
2. Lab reminder: Th 7-8PM[LA01] / 6-7PM[LA02]



- Slides credit: some slides are adapted from Haiyun He (Cornell).
- Note: Note of 6-867 (MIT)
  Sections 8-9 in Note of CS299 (Stanford)
  Webpage: (online note)
  Book: 'Pattern Recognition and Machine Learning' by Christopher M. Bishop

